

Cost of Software-Related Disasters

“To err is human, but to really foul things up you need a computer.” –Paul Ehrlich

- Software errors cost the U.S. economy \$60 billion annually in rework, lost productivity and actual damages. We all know software bugs can be annoying, but faulty software can also be expensive, embarrassing, destructive and deadly.

1. Mariner Bugs Out (1962)



Cost: \$18.5 million

Disaster: The Mariner 1 rocket with a space probe headed for Venus diverted from its intended flight path shortly after launch. Mission Control destroyed the rocket 293 seconds after liftoff.

Cause: A programmer incorrectly transcribed a handwritten formula into computer code, missing a single superscript bar. Without the smoothing function indicated by the bar, the software treated normal variations of velocity as if they were serious, causing faulty corrections that sent the rocket off course.

2. Hartford Coliseum Collapse (1978)



Cost: \$70 million, plus another \$20 million damage to the local economy

Disaster: Just hours after thousands of fans had left the Hartford Coliseum, the steel-latticed roof collapsed under the weight of wet snow.

Cause: The programmer of the CAD software used to design the coliseum incorrectly assumed the steel roof supports would only face pure compression. But when one of the supports unexpectedly buckled from the snow, it set off a chain reaction that brought down the other roof sections like dominoes.

3. CIA Gives the Soviets Gas (1982)

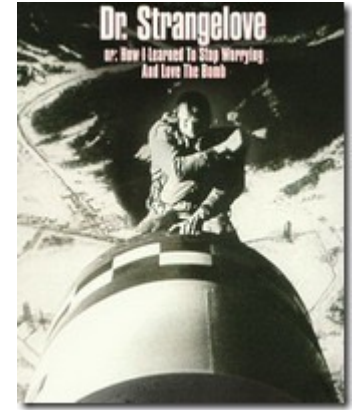


Cost: Millions of dollars, significant damage to Soviet economy

Disaster: Control software went haywire and produced intense pressure in the Trans-Siberian gas pipeline, resulting in the largest man-made non-nuclear explosion in Earth's history.

Cause: CIA operatives allegedly planted a bug in a Canadian computer system purchased by the Soviets to control their gas pipelines. The purchase was part of a strategic Soviet plan to steal or covertly obtain sensitive U.S. technology. When the CIA discovered the purchase, they sabotaged the software so that it would pass Soviet inspection but fail in operation.

4. World War III... Almost (1983)



Cost: Nearly all of humanity

Disaster: The Soviet early warning system falsely indicated the United States had launched five ballistic missiles. Fortunately the Soviet duty officer had a “funny feeling in my gut” and reasoned if the U.S. was really attacking they would launch more than five missiles, so he reported the apparent attack as a false alarm.

Cause: A bug in the Soviet software failed to filter out false missile detections caused by sunlight reflecting off cloud-tops.

5. Medical Machine Kills (1985)



- **Cost:** Three people dead, three people critically injured
- **Disaster:** Canada's Therac-25 radiation therapy machine malfunctioned and delivered lethal radiation doses to patients.
- **Cause:** Because of a subtle bug called a race condition, a technician could accidentally configure Therac-25 so the electron beam would fire in high-power mode without the proper patient shielding.

6. Wall Street Crash (1987)

Cost: \$500 billion in one day

Disaster: On “Black Monday” (October 19, 1987), the Dow Jones Industrial Average plummeted 508 points, losing 22.6% of its total value. The S&P 500 dropped 20.4%. This was the greatest loss Wall Street ever suffered in a single day.

Cause: A long bull market was halted by a rash of SEC investigations of insider trading and by other market forces. As investors fled stocks in a mass exodus, computer trading programs generated a flood of sell orders, overwhelming the market, crashing systems and leaving investors effectively blind.



7. AT&T Lines Go Dead (1990)

Cost: 75 million phone calls missed, 200 thousand airline reservations lost

Disaster: A single switch at one of AT&T's 114 switching centers suffered a minor mechanical problem and shut down the center. When the center came back up, it sent a message to other switching centers, which in turn caused them to shut down and brought down the entire AT&T network for 9 hours.

Cause: A single line of buggy code in a complex software upgrade implemented to speed up calling caused a ripple effect that shut down the network.



8. Patriot Fails Soldiers (1991)

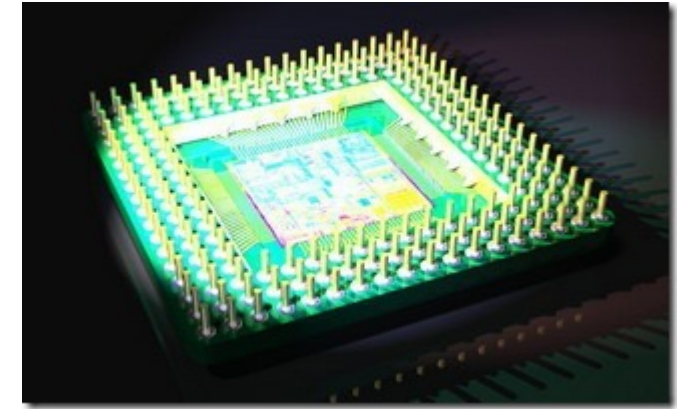
Cost: 28 soldiers dead, 100 injured

Disaster: During the first Gulf War, an American Patriot Missile system in Saudi Arabia failed to intercept an incoming Iraqi Scud missile. The missile destroyed an American Army barracks.

Cause: A software rounding error incorrectly calculated the time, causing the Patriot system to ignore the incoming Scud missile.



9. Pentium Fails Long Division (1993)

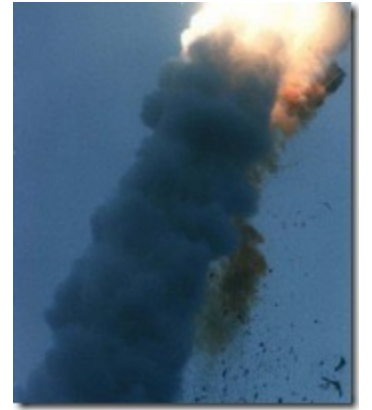


Cost: \$475 million, corporate credibility

Disaster: Intel's highly-promoted Pentium chip occasionally made mistakes when dividing floating-point numbers within a specific range. For example, dividing $4195835.0/3145727.0$ yielded 1.33374 instead of 1.33382, an error of 0.006%. Although the bug affected few users, it became a public relations nightmare. With an estimated 5 million defective chips in circulation, Intel offered to replace Pentium chips only for consumers who could prove they needed high accuracy. Eventually Intel replaced the chips for anyone who complained.

Cause: The divider in the Pentium floating point unit had a flawed division table, missing about five of a thousand entries and resulting in these rounding errors.

10. Ariane Rocket Goes Boom (1996)



Cost: \$500 million

Disaster: Ariane 5, Europe's newest unmanned rocket, was intentionally destroyed seconds after launch on its maiden flight. Also destroyed was its cargo of four scientific satellites to study how the Earth's magnetic field interacts with solar winds.

Cause: Shutdown occurred when the guidance computer tried to convert the sideways rocket velocity from 64-bits to a 16-bit format. The number was too big, and an overflow error resulted. When the guidance system shut down, control passed to an identical redundant unit, which also failed because it was running the same algorithm.

11. Skynet Brings Judgement Day (1997)



Cost: 6 billion dead, near-total destruction of human civilization and animal ecosystems (fictional)

Disaster: Human operators attempt to shut off the Skynet global computer network. Skynet responds by firing U.S. nuclear missiles at Russia, initiating global nuclear war on what became known as Judgement Day (August 29, 1997).

Cause: Cyberdyne, the leading weapons manufacturer, installed Skynet technology in all military hardware including stealth bombers and missile defense systems. The Skynet technology formed a seamless network and effectively removed humans from strategic defense. Eventually Skynet became sentient, was threatened when the humans tried to take it offline, sought to survive, and retaliated with nuclear war.

12. Mars Climate Crasher (1998)



Cost: \$125 million

Disaster: After a 286-day journey from Earth, the Mars Climate Orbiter fired its engines to push into orbit around Mars. The engines fired, but the spacecraft fell too far into the planet's atmosphere, likely causing it to crash on Mars.

Cause: The software that controlled the Orbiter thrusters used imperial units (pounds of force), rather than metric units (Newtons) as specified by NASA.

13. Disastrous Study (1999)



Cost: Scientific credibility

Disaster: In this ironic case, software used to analyze disasters had a disaster of its own. The *New England Journal of Medicine* reported increased suicide rates after severe natural disasters. Unfortunately, these results proved to be incorrect.

Cause: A programming error caused the number of suicides for one year to be doubled, which was enough to throw off the entire study.

14. British Passports to Nowhere (1999)



Cost: £12.6 million, mass inconvenience

Disaster: The U.K. Passport Agency implemented a new Siemens computer system, which failed to issue passports on time for a half million British citizens. The Agency had to pay millions in compensation, staff overtime and umbrellas for people queuing in the rain for passports.

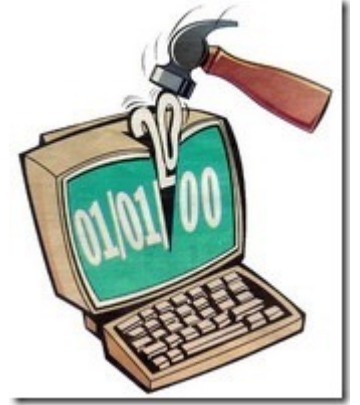
Cause: The Passport Agency rolled out its new computer system without adequately testing it or training its staff. At the same time, a law change required all children under 16 traveling abroad to obtain a passport, resulting in a huge spike in passport demand that overwhelmed the buggy new computer system.

15. Y2K (1999)

Cost: \$500 billion

Disaster: One man's disaster is another man's fortune, as demonstrated by the infamous Y2K bug. Businesses spent billions on programmers to fix a glitch in legacy software. While no significant computer failures occurred, preparation for the Y2K bug had a significant cost and time impact on all industries that use computer technology.

Cause: To save computer storage space, legacy software often stored the year for dates as two digit numbers, such as "99" for 1999. The software also interpreted "00" to mean 1900 rather than 2000, so when the year 2000 came along, bugs would result.



16. Dot-Bomb Collapse (2000)



Cost: \$5 trillion in market value, thousands of companies failed

Disaster: A speculative bubble from 1995–2001 fueled a rapid increase in venture capital investments and stock market values in the Internet and technology sectors. The “dot-com bubble” began to collapse in early 2000, erasing trillions in stock market value, wiping out thousands of companies and jobs, and launching a global recession.

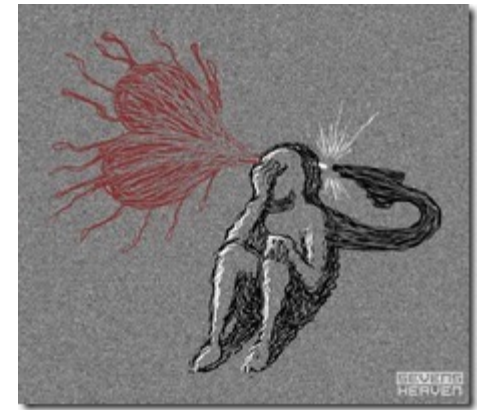
Cause: Companies and investors dismissed standard business models, and instead focused on increasing market share at the expense of profits.

17. Love Virus (2000)

Cost: [\\$8.75 billion](#), millions of computers infected, significant data loss

Disaster: The LoveLetter worm infected millions of computers and caused more damage than any other computer virus in history. The worm deleted files, changed home pages and messed with the Registry.

Cause: LoveLetter infected users via e-mail, Internet chat and shared file systems. The email had an executable file attachment and subject line, “ILOVEYOU.” When the user opened the attachment, the virus would infect the user’s computer and send itself to everyone in the address book.



18. Cancer Treatment to Die For (2000)



Cost: Eight people dead, 20 critically injured

Disaster: Radiation therapy software by Multidata Systems International miscalculated the proper dosage, exposing patients to harmful and in some cases fatal levels of radiation. The physicians, who were legally required to double-check the software's calculations, were indicted for murder.

Cause: The software calculated radiation dosage based on the order in which data was entered, sometimes delivering a double dose of radiation.

19. EDS Drops Child Support (2004)

Cost: £539 million and counting

Disaster: Business services giant EDS developed a computer system for U.K.'s Child Support Agency (CSA) that accidentally overpaid 1.9 million people, underpaid another 700,000, had £3.5 billion in uncollected child support payments, a backlog of 239,000 cases, 36,000 new cases “stuck” in the system, and still over 500 documented bugs.

Cause: EDS introduced a large, complex IT system to the CSA while trying to simultaneously restructure the agency.



20. FBI's Trilogy Terminated (2005)

Cost: \$105 million, still no effective case file solution

Disaster: The FBI scrapped its computer systems overhaul after four years of effort. The Virtual Case File project was a massive, integrated software system for agents to share case files and other information.

Cause: Mismanagement, and an attempt to build a long-term project on technology that was outdated before the project completed, resulted in a complex and unusable system.



What if we don't have sufficient software development and management capabilities?

"Software is devouring the world."

These words accurately describe the current situation facing companies. As we enter the intelligent information society represented by Industry 4.0, software has emerged as a key driver of digital transformation and enterprises have their competitors strained in the industry by creating a new business model that did not exist before the widespread adoption of software. We can see evidence for this in the way that Uber and Airbnb have established themselves as the providers of the world's largest transportation service and its largest accommodation service, respectively, even though they do not own any cars or hotels. As a result, many enterprises all over the world are trying to strengthen their software capabilities for digital business. The impact is so far-reaching that even the manufacturing company GE and the financial company Goldman Sachs also see themselves as software companies. We can easily find evidence of the newly elevated status of software all around us. Artificial intelligence (AI) is the most representative example. "AlphaGo", which was introduced to the world by Google DeepMind a few years ago, became an important measure by which we can estimate the extent to which the power of software will develop. At present, AI enriches our lives considerably as it is utilized in diverse fields in our daily lives, such as communication, education, shopping, entertainment, vehicle management, etc.

Autonomous driving, which has been actively researched and developed throughout the world, also belongs to the software domain. Until very recently, mechanical engineering elements had a dominant place in the car and electric apparatus industries, which were even referred to as the "flowers of the manufacturing sector". However, more than half of the industries are now composed of software code. In particular, the car industry depends increasingly heavily on software as self-driving technology is increasingly incorporated into new models.

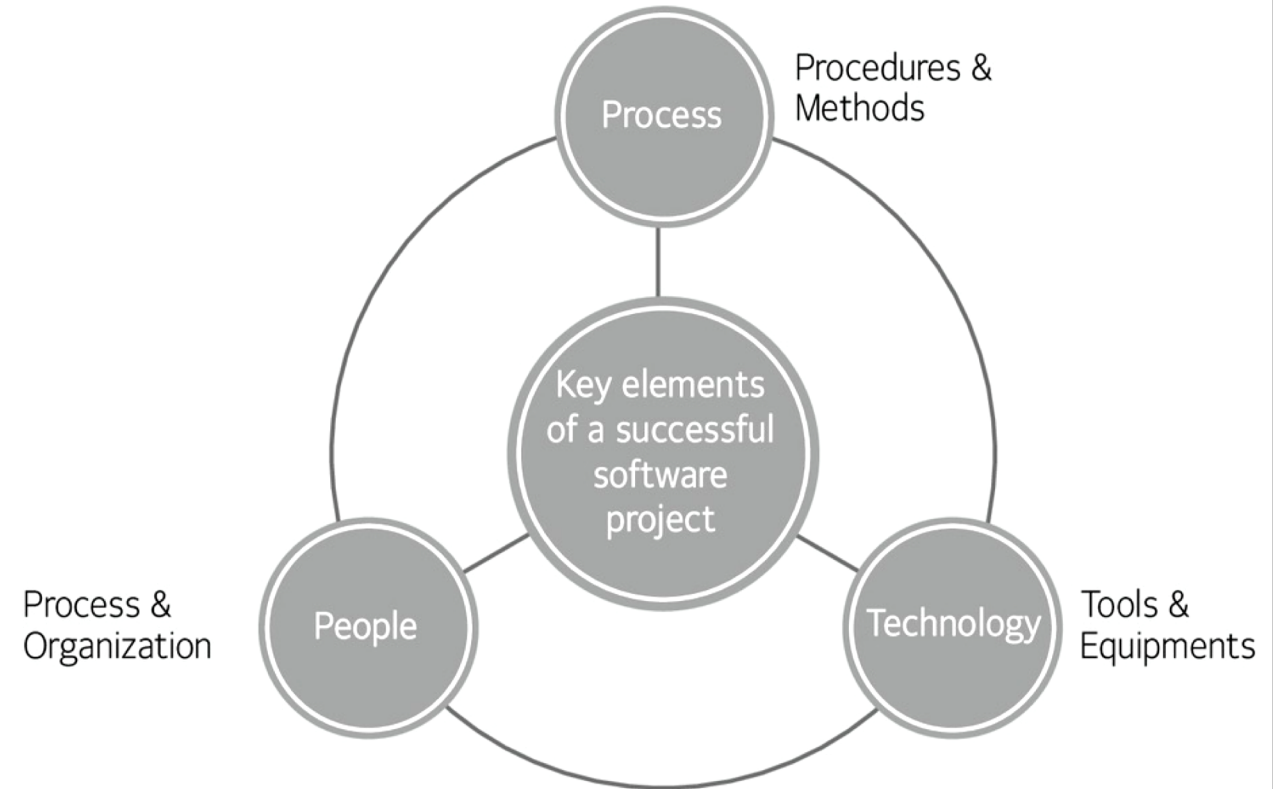
Even though such important software is increasingly being used in more areas, the pertinent question is whether software quality is being managed, and productivity being improved, in proportion to the increased use. In particular, it is necessary to examine whether software functions are working properly as the use of software is increasing even in areas directly related to human life or property, such as autonomous driving. However, the reality is that software quality cannot be considered in the domestic software industry. For example, more mobile apps are now used as smartphones have become so popular. However, these mobile apps are not used by a single user but rather are called by several users at the same time. Therefore, unintended defects can occur while seeking to improve service efficiency.

As such, even though software is prone to be defective due to its intrinsic nature, the domestic software industry tends to overlook it on such pretexts as a lack of budget or a shortage of manpower. One can understand their view that they don't have any other options. It is natural that the software development period and cost increase when quality is taken into account. This phenomenon is further aggravated by continuous issues that have been pointed out for a long time, but which still have not been resolved, namely, the difficulty of getting the right software price, the lack of developers, a development culture centered on SI projects, etc. [Source] "Quality improvement is essential in the software-centered era." - <https://blog.naver.com/rsupport/221358144195>

In the past, software development projects could be conducted normally without systematic implementation simply by relying on the experience of programmers. However, today's large projects involving hundreds of developers face many challenges, such as communication, schedule and cost management issues that arise from the lengthy development period and changes to ambiguous and complex requirements, etc. Therefore, it is essential to understand how to develop and manage software systematically and to improve enterprises' competencies, in order to develop software of the highest quality and promote productivity.

Background and Purpose of Software Engineering

- To effectively apply software engineering technology, it is necessary to prepare and three key elements in a balanced and harmonious manner, and make continuous effort to maintain them.



Background to Software Engineering

- 1950s:
 - Concept of software engineering, like hardware engineering, began to be introduced in software development projects
- 1960s:
 - “Software crisis”
 - “Coding first and correction later”
 - Formal techniques were less usable to general software developers
 - Waterfall method was expensive and slow to implement

Background to Software Engineering

- 1990s:
 - Software vendors had to reduce “time to market” to gain competitive advantage
 - Researches were done on how to raise software development productivity, and models focusing on **concurrent engineering** were adopted.
- 2000s:
 - Agile methodology was comprehensively adopted to effectively respond to rapid changes in the software market

Four Key Elements of Software Engineering

- **Software Engineering:** A discipline that studies the overall cycle of software – such as the development, operation, and maintenance – systematically, descriptively, and quantitatively.
- Four Key Elements:
 1. Method
 2. Tool
 3. Procedure
 4. People

Method

- The method is composed of project planning and estimation, system and software analysis, data structure, program structure, algorithm, coding, testing, and maintenance tasks.
- Methods centered on a specific language (e.g. an object-oriented method), or graphical notation, are introduced from time to time.
- A series of evaluation standards for software quality is introduced.

Tool

- “Tool” refers to an automated or semi-automated method that is used to improve productivity or consistency when performing a task.
- There are numerous tools in the software development lifecycle (e.g. requirements management tools, modeling tools, configuration management tools, change management tools).
- When tools are integrated in such a way that the information created by one tool can be used by other tools, it is set as a system that supports software development.

Procedure

- A procedure combines a method and a tool so that they can be used to develop software in a rational and timely fashion.
- The procedure defines the applied method, required deliverables (documents, reports, etc.), controls to guarantee quality and assist coordinate adjustment, and the sequence of milestones that enable software managers to evaluate progress.

People

- As many tasks (establishment, improvement, maintenance, etc.) are performed by employees and organizations specializing in software engineering, software engineering relatively depends more heavily on people.
- It is practically impossible to summarize software development in an easily accessible or graspable manner

Software Development Life Cycle

- Requirements Analysis
- Design
- Implementation and Unit Testing
- Integration
- System Testing
- Deployment and Maintenance

System Engineering

- System: purposeful collection of interrelated components that work together to achieve some objective.
- Computer-based systems: consists of hardware, software, data, operational processes and organizational policies, people and documentation.

System Life Cycle

System Life Cycle

- System Requirements Analysis
- System design
- Subsystem development
(software is considered one of these subsystems)
- System integration
- System Testing
- Deployment and Maintenance

Software Development Life Cycle

- Requirements Analysis
- Design
- Implementation and Unit Testing
- Integration
- System Testing
- Deployment and Maintenance

Seatwork: Groups of 2-3 members

Reflect on broader implications of Software Disasters:

1. What do these cases reveal about the importance of software engineering as a discipline? (You may cite previous examples mentioned in class)
2. How does it connect to today's world, where software is even more deeply embedded in society (medicine, transportation, defense, finance, etc.)?